

FUNCTION

In **Python**, a **function** is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, you can define it once and call it whenever needed.

◆ 1. What is a Function?

A function is like a mini-program inside your program. It:

- Takes input (optional)
 - Processes it
 - Returns output (optional)
-

◆ 2. Defining a Function

You define a function using the `def` keyword:

```
def greet():  
    print("Hello, world!")
```

Explanation:

- `def` → keyword to define a function
 - `greet` → function name
 - `()` → parameters (empty here)
 - `:` → start of function body
-

◆ 3. Calling a Function

To use a function:

```
greet()
```

👉 Output:

```
Hello, world!
```

◆ 4. Function with Parameters

Functions can take inputs:

```
def greet(name):  
    print("Hello,", name)  
greet("Alice")
```

👉 Output:

Hello, Alice

◆ 5. Function with Return Value

Functions can return results using return:

```
def add(a, b):
```

```
    return a + b
```

```
result = add(5, 3)
```

```
print(result)
```

👉 Output:

8

◆ 6. Types of Functions

✓ Built-in Functions

Already available in Python:

print()

len()

type()

✓ User-defined Functions

Functions you create:

```
def my_function():
```

```
    pass
```

◆ 7. Default Parameters

You can give default values:

```
def greet(name="Guest"):
```

```
    print("Hello,", name)
```

```
greet()
```

```
greet("John")
```

◆ 8. Keyword Arguments

You can pass arguments by name:

```
def student(name, age):  
    print(name, age)
```

```
student(age=20, name="Sam")
```

◆ 9. Variable-Length Arguments

***args (multiple positional arguments)**

```
def add_numbers(*numbers):  
    return sum(numbers)
```

```
print(add_numbers(1, 2, 3, 4))
```

****kwargs (multiple keyword arguments)**

```
def details(**info):  
    print(info)
```

```
details(name="Alice", age=25)
```

◆ 10. Lambda (Anonymous Functions)

Short one-line functions:

```
square = lambda x: x * x  
print(square(4))
```

◆ 11. Scope of Variables

- **Local variable** → inside function
- **Global variable** → outside function

```
x = 10 # global
```

```
def test():
```

```
    x = 5 # local
```

```
    print(x)
```

```
test()
```

```
print(x)
```

👉 Output:

```
5
```

```
10
```

◆ 12. Why Use Functions?

Functions help:

- Avoid repetition (DRY principle)
- Make code readable
- Break large problems into smaller parts
- Improve debugging and testing

◆ 13. Example Program

```
def calculate_area(length, width):
```

```
    return length * width
```

```
area = calculate_area(5, 4)
```

```
print("Area:", area)
```

If you want, I can also show **real-world examples**, **interview questions**, or **practice problems** to master Python functions.

Link : CHATGPT

Lambda Function in Python (Detailed Explanation)

A **Lambda function** in **Python** is a small **anonymous function** (a function without a name).

It is used when you need a simple function for a short period of time.

1. Syntax of Lambda Function

lambda arguments: expression

Structure

- lambda → keyword used to create a lambda function
- arguments → input values
- expression → operation to perform
- The result of the expression is automatically returned.

General form:

```
lambda x: x + 10
```

This means:

- Take x as input.
 - Add 10 to it.
 - Return the result.
-

2. Normal Function vs Lambda Function

Normal Function

```
def square(x):  
    return x * x
```

```
print(square(5))
```

Output:

```
25
```

Lambda Function

```
square = lambda x: x * x
```

```
print(square(5))
```

Output:

25

Both give the same result.

3. Lambda with One Argument

```
double = lambda x: x * 2
```

```
print(double(6))
```

Output:

12

Explanation

Input:

x = 6

Expression:

6 * 2

Result:

12

4. Lambda with Multiple Arguments

Two Arguments

```
add = lambda a, b: a + b
```

```
print(add(10, 20))
```

Output:

```
30
```

Three Arguments

```
multiply = lambda a, b, c: a * b * c
```

```
print(multiply(2, 3, 4))
```

Output:

```
24
```

5. Immediately Invoked Lambda Function

You can create and call it at the same time.

```
print((lambda x: x + 5)(10))
```

Output:

```
15
```

Explanation

Lambda:

```
lambda x: x + 5
```

Input:

```
10
```

Calculation:

```
10 + 5
```

Result: 15

6. Lambda with `map()`

`map()` applies a function to every element of an iterable.

Example

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(map(lambda x: x * 2, numbers))
```

```
print(result)
```

Output:

```
[2, 4, 6, 8, 10]
```

How it works

Element	Lambda	Result
1	1×2	2
2	2×2	4
3	3×2	6
4	4×2	8
5	5×2	10

7. Lambda with `filter()`

`filter()` selects elements based on a condition.

Example

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(even)
```

Output:

```
[2, 4, 6]
```

How it works

Condition:

```
x % 2 == 0
```

Number Condition Keep?

1	False	No
2	True	Yes
3	False	No
4	True	Yes
5	False	No
6	True	Yes

8. Lambda with `sorted()`

Lambda is commonly used as a sorting key.

Example

```
students = [  
    ("John", 85),  
    ("Alice", 92),  
    ("Bob", 78)  
]
```

```
sorted_students = sorted(students,  
                          key=lambda x: x[1])
```

```
print(sorted_students)
```

Output:

```
[('Bob', 78), ('John', 85), ('Alice', 92)]
```

Explanation

`x[1]` means:

- ("John", 85) → 85
- ("Alice", 92) → 92
- ("Bob", 78) → 78

Sorting happens based on marks.

Descending Order

```
sorted_students = sorted(  
    students,  
    key=lambda x: x[1],  
    reverse=True  
)
```

```
print(sorted_students)
```

Output:

```
[('Alice', 92), ('John', 85), ('Bob', 78)]
```

9. Lambda with `reduce()`

`reduce()` is available in the `functools` module.

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(lambda x, y: x + y, numbers)
```

```
print(result)
```

Output:

```
10
```

How it works

$1 + 2 = 3$

$3 + 3 = 6$

$6 + 4 = 10$

10. Limitations of Lambda Functions

Lambda can contain only one expression.

✓ Valid:

```
lambda x: x + 1
```

✗ Invalid:

```
lambda x:  
    y = x + 1
```

```
return y
```

You cannot use:

- Multiple statements
- Loops
- return
- Assignments

inside a lambda function.

11. When Should You Use Lambda?

Use lambda when:

- ✓ The function is short and simple.
- ✓ You need it only once.
- ✓ Using map(), filter(), sorted(), or reduce().

Example:

```
sorted(names, key=lambda name: len(name))
```

12. When Should You Avoid Lambda?

Avoid lambda when:

- ✗ The logic is complex.
- ✗ Multiple statements are required.
- ✗ The function will be reused many times.

In such cases, use a normal function:

```
def calculate_discount(price, discount):  
    final_price = price - (price * discount / 100)  
    return final_price
```

Summary

Feature	Normal Function	Lambda Function
Name	Yes	Usually No
Keyword	def	lambda
Multiple Statements	Yes	No
return Keyword	Required	Automatic
Best For	Complex logic	Short operations
Reusability	High	Usually temporary

Remember

Lambda = Small, anonymous, single-expression function used for short and simple tasks.

Link : [CHATGPT](#)

map() Function in Python (Detailed Explanation)

The `map()` function in **Python** is used to apply a function to every item of an iterable (such as a list, tuple, or set).

Instead of using a loop to process each element one by one, `map()` does it automatically.

1. Syntax

```
map(function, iterable)
```

Parameters

- **function** → The function that will be applied to each element.
- **iterable** → The collection of elements (list, tuple, set, etc.).

Returns

`map()` returns a **map object** (an iterator).

2. Simple Example

Without `map()`:

```
numbers = [1, 2, 3, 4]

result = []

for n in numbers:
    result.append(n * 2)

print(result)
```

Output:

```
[2, 4, 6, 8]
```

Using `map()`:

```
numbers = [1, 2, 3, 4]

result = map(lambda x: x * 2, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4, 6, 8]
```

3. How `map()` Works

Example:

```
numbers = [1, 2, 3]
result = map(lambda x: x + 10, numbers)
print(list(result))
```

Output:

```
[11, 12, 13]
```

Step-by-Step

Element	Lambda Function	Result
1	$1 + 10$	11
2	$2 + 10$	12
3	$3 + 10$	13

4. Using a Normal Function

You don't have to use lambda.

```
def square(x):
    return x * x

numbers = [1, 2, 3, 4]
result = map(square, numbers)
print(list(result))
```

Output:

```
[1, 4, 9, 16]
```

5. Why `list()` is Used?

`map()` returns a map object.

```
numbers = [1, 2, 3]
result = map(lambda x: x * 2, numbers)
print(result)
```

Output:

```
<map object at 0x...>
```

To see the actual values:

```
print(list(result))
```

Output:

```
[2, 4, 6]
```

6. Using `map()` with Multiple Iterables

`map()` can process two or more iterables at the same time.

Example

```
list1 = [1, 2, 3]
list2 = [10, 20, 30]
result = map(lambda x, y: x + y, list1, list2)
print(list(result))
```

Output:

```
[11, 22, 33]
```

How It Works

	list1	list2	x + y
1	1	10	11
2	2	20	22
3	3	30	33

7. Different Length Iterables

`map()` stops when the shortest iterable ends.

```
list1 = [1, 2, 3, 4]
list2 = [10, 20]

result = map(lambda x, y: x + y, list1, list2)

print(list(result))
```

Output:

```
[11, 22]
```

Only two results are produced because `list2` has only two elements.

8. Converting Data Types Using `map()`

String to Integer

```
numbers = ["1", "2", "3", "4"]

result = map(int, numbers)

print(list(result))
```

Output:

```
[1, 2, 3, 4]
```

Integer to String

```
numbers = [1, 2, 3]

result = map(str, numbers)

print(list(result))
```

Output:

```
['1', '2', '3']
```

9. Convert Names to Uppercase

```
names = ["krish", "amit", "rahul"]

result = map(str.upper, names)

print(list(result))
```


Output:

```
['KRISH', 'AMIT', 'RAHUL']
```

10. Using `map()` with Tuples

```
numbers = (1, 2, 3, 4)

result = map(lambda x: x * 5, numbers)

print(list(result))
```

Output:

```
[5, 10, 15, 20]
```

11. `map()` vs `for` Loop

Feature	<code>map()</code>	<code>for</code> Loop
Code Length	Short	Longer
Speed	Usually faster	Slightly slower
Readability	Good for simple operations	Better for complex logic
Multiple Iterables	Yes	Yes
Complex Conditions	Not ideal	Better

12. When to Use `map()`

Use `map()` when:

- ✅ The same operation must be performed on every element.
- ✅ The operation is simple.
- ✅ You want concise code.

Example:

```
prices = [100, 200, 300]

gst_prices = map(lambda x: x * 1.18, prices)

print(list(gst_prices))
```

Output:

```
[118.0, 236.0, 354.0]
```

13. When NOT to Use `map()`

Avoid `map()` when:

- ✗ The logic is complicated.
- ✗ Multiple statements are needed.
- ✗ Readability becomes difficult.

Use a `for` loop instead.

Summary

```
map(function, iterable)
```

- Applies a function to each element.
- Returns a map object.
- Convert to `list()` to view results.
- Can work with multiple iterables.
- Often used with `lambda`.

Example

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x ** 2, numbers))
print(result)
```

Output:

```
[1, 4, 9, 16]
```

Easy Formula to Remember

`map()` = Apply one function to all elements of an iterable and return the transformed values.

filter() Function in Python (Detailed Explanation)

The **filter()** function in **Python** is used to **select elements from an iterable based on a condition**.

It keeps only those elements for which the condition returns True.

1. Syntax

`filter(function, iterable)`

Parameters

- **function** → A function that tests each element and returns True or False.
- **iterable** → The collection (list, tuple, set, etc.) whose elements you want to filter.

Returns

`filter()` returns a **filter object** (an iterator).

2. Simple Example

Suppose we want only even numbers from a list.

Without `filter()`:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even = []
```

```
for n in numbers:
```

```
    if n % 2 == 0:
```

```
        even.append(n)
```

```
print(even)
```

Output:

```
[2, 4, 6]
```

Using `filter()`:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = filter(lambda x: x % 2 == 0, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4, 6]
```

3. How filter() Works

Example:

```
numbers = [1, 2, 3, 4]
```

```
result = filter(lambda x: x > 2, numbers)
```

```
print(list(result))
```

Output:

```
[3, 4]
```

Step-by-Step

	Element	Condition ($x > 2$)	Keep?
1	False		No
2	False		No
3	True		Yes
4	True		Yes

Only elements with True are kept.

4. Using a Normal Function

You don't have to use lambda.

```
def is_even(x):
```

```
    return x % 2 == 0
```

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = filter(is_even, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4, 6]
```

5. Why list() is Used?

filter() returns a filter object.

```
numbers = [1, 2, 3]
```

```
result = filter(lambda x: x > 1, numbers)
```

```
print(result)
```

Output:

```
<filter object at 0x...>
```

To see the actual values:

```
print(list(result))
```

Output:

```
[2, 3]
```

6. Filtering Odd Numbers

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
odd = filter(lambda x: x % 2 != 0, numbers)
```

```
print(list(odd))
```

Output:

```
[1, 3, 5]
```

7. Filtering Positive Numbers

```
numbers = [-5, -2, 0, 3, 7, -1]
```

```
positive = filter(lambda x: x > 0, numbers)
```

```
print(list(positive))
```

Output:

```
[3, 7]
```

8. Filtering Strings

Suppose we want names longer than 4 characters.

```
names = ["Krish", "Amit", "Rahul", "Jay"]  
result = filter(lambda name: len(name) > 4, names)  
print(list(result))
```

Output:

```
['Krish', 'Rahul']
```

9. Using filter() with None

If the function is None, filter() removes all **falsy values**.

Falsy values include:

- 0
- False
- None
- "" (empty string)
- [] (empty list)
- {} (empty dictionary)

Example:

```
data = [0, 1, False, True, "", "Hello", None]  
result = filter(None, data)  
print(list(result))
```

Output:

```
[1, True, 'Hello']
```

10. filter() vs map()

filter()

Selects elements based on a condition.

```
numbers = [1, 2, 3, 4]
```

```
result = filter(lambda x: x > 2, numbers)
```

```
print(list(result))
```

Output:

```
[3, 4]
```

map()

Transforms every element.

```
numbers = [1, 2, 3, 4]
```

```
result = map(lambda x: x * 2, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4, 6, 8]
```

Difference Table

Feature	filter()	map()
Purpose	Select elements	Transform elements
Function Returns	True/False	Any value
Number of Output Elements	Same or fewer	Usually same
Main Use	Conditions	Modifications

11. filter() vs for Loop

Using filter()

```
numbers = [1, 2, 3, 4]
```

```
result = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(result)
```

Using for Loop

```
numbers = [1, 2, 3, 4]
```

```
result = []
```

```
for n in numbers:
```

```
    if n % 2 == 0:
```

```
        result.append(n)
```

```
print(result)
```

Both produce:

```
[2, 4]
```

12. When to Use filter()

Use filter() when:

- ✓ You need elements satisfying a condition.
- ✓ The condition is simple.
- ✓ You want concise code.

Example:

```
marks = [35, 80, 25, 90, 50]
```

```
passed = filter(lambda x: x >= 35, marks)
```

```
print(list(passed))
```

Output:

```
[35, 80, 90, 50]
```

13. When NOT to Use filter()

Avoid filter() when:

- ✗ The logic is complex.

✗ Multiple conditions make the code difficult to read.

✗ A normal for loop is clearer.

Summary

`filter(function, iterable)`

- Checks each element using a condition.
- Keeps elements for which the condition is True.
- Returns a filter object.
- Convert it using `list()` to see the values.
- Commonly used with `lambda`.

Easy Formula to Remember

`filter()` = Keep only the elements that satisfy a condition.

Example

```
numbers = [10, 15, 20, 25, 30]
```

```
result = list(filter(lambda x: x % 10 == 0, numbers))
```

```
print(result)
```

Output:

```
[10, 20, 30]
```

Quick Memory Trick

- `map()` → Modify every element
- `filter()` → Select some elements
- `reduce()` → Combine all elements into one result

Link : [CHATGPT](#)

Link : [ALL](#)